Royal vs Frugal Amp

Coding Agent Benchmark: a workflow demo with Amp, not a scientific benchmark.



Zain Fathoni

Senior Software Engineer at BookThatApp. Backend, manager, frontend, now fullstack.

Agentic engineering practitioner · SWE Growth Webinar, 25 October 2026.

https://zainfathoni.com

Not everyone has the same budget

Spending styles, not skill levels:

- **Sultan Token** — spends freely on the best models and hosted compute
- **Mendang-mending** — balances cost and quality
- **Kere Hore** — keeps cost low and still wants real results

Royal

My personal label: frontier models plus Amp-hosted Orbs under a Megawatt subscription with Orb quota. Not an official Amp mode.

Frugal

My personal label for today: cheaper worker models, runner-only, no Orbs.

P3 · COMING PROBLEM

Cost has two parts

Model cost: tokens and credits for each model call.

Compute cost: where the agent runs, and the Orb quota it uses.

Runner-only is not free inference. Your own machine saves Orb quota. Model calls still cost money.

🧠 Model cost

Planner, workers, reviewers, and rescue calls. Each one has its own usage record.

🖥️ Compute cost

Amp-hosted Orbs, self-hosted infrastructure, or your own runner.

P4 · EMOTIONAL WIN

Useful, verified work within budget

Today's goal: finish real tasks with a cheaper worker model.

Each task needs **checks** and **evidence**. A claim is not proof.

✅ A task counts as done when:

the combined code passes typecheck and build, the acceptance checks pass, and the usage record is filled in.

⚠️ We do not know the result yet.

The demo decides. This slide states the bar, not the outcome.

P5 · FALSE HOPE

One cheap model is not parallel work

A giant prompt does not split the work for you.

More worktrees do not add ports, RAM, or browser sessions.

Three cheap workers can still hit the **same file, port, or test database.**

📜 One giant prompt

Mixes three tasks. One worker loses track, and nobody owns the result.

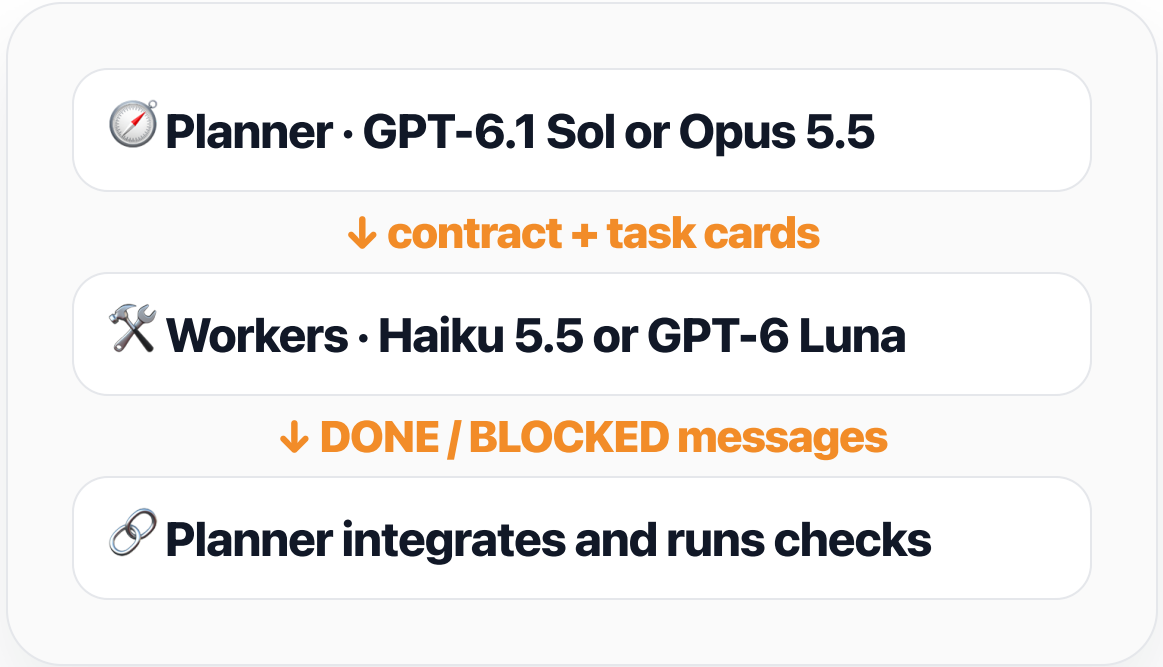
🌳 Three worktrees

Separate source edits. The same port 3000 for all three.

Plan high, run bounded, connect threads

Amp's standout feature: **threads can message each other.** A planner can coordinate cheaper workers on a runner.

Messages carry context, not code. Code still flows through branches and checks.



Four rules make workers safe

Small scope, fixed contracts, owned resources, and checks written first.

You already know how to do this. The change is to write it down before the workers start.

🎯 Scope

One task. A short file list.

📐 Contracts

Fixed names and data shapes.

🔒 Resource ownership

Own port, data, browser profile.

🧪 Checks

Define acceptance checks first.

Start a runner-only parallel demo

1. Use one runner. No Orbs.
2. Planner writes the contract and three task cards.
3. Start three bounded workers, one worktree each.
4. Give each worker its own port and test data.
5. Wait for messages, then integrate on one branch.

> **Parallel work needs owners, not just more agents.**

Rule for today's run.

Observe the integrated result

- changed files for each task
- check results for each task
- a usage record for each thread
- a blocked worker shows where the contract was weak

We will not claim success early.

We show the combined result only after the checks run.

Failures count too.

A blocked worker is useful evidence. It points to a weak contract.

Choose what is worth paying for

> **Saving does not mean everything must be cheap. Saving means knowing which parts are worth paying for.**

Pay for planning, review, and rescue where they matter. Use cheaper workers where the task is bounded.

Where to spend more

Scope, contracts, review of combined work, and rescue of blocked tasks.

Where to spend less

Bounded edits, repeated checks, and first drafts that a check can reject.

Today’s run order

TIME (WIB)	BLOCK	OUTPUT
13.00–13.15	Story, P1–P10	Shared frame
13.15–13.25	Contract and task cards	Contract v1
13.25–14.10	Workers run, planner answers messages	Branches and messages
14.10–14.15	Integration and checks	Combined result
14.15–14.30	Q&A	Questions

Backup if a worker is slow:

Show the planner's message log and the blocked state. Do not hide it.

Live Demo

Runner only. No Orbs. Three bounded tasks in one repository.

DEMO RULES

What we do not do today

- no Orbs, and no self-hosted Orbs
- no production writes and no deploys
- no new Cloudflare resources without my approval
- no shared port, database, or browser profile between workers
- no result claimed before its check runs

Why these rules?

They keep the run repeatable and make the cost record clear.

COST MODEL

What we record for each layer

LAYER	ROLE	MODEL CHOICE	RECORD
Planner	scopes tasks and reviews	GPT-6.1 Sol or Sonnet/Opus 5.5	usage per thread
Workers	bounded edits and checks	Haiku 5.5 or GPT-6 Luna	usage per thread
Rescue	only when a worker is blocked	planner model	usage per thread
Compute	runner machine	own runner, no Orbs	runner time
Human	your review time	none	count it, not billed

No prices on this slide. We fill in usage from Amp usage records after the run.

SETUP

Models and runner

- **Planner:** Amp mode `gpt61s` (GPT-6.1 Sol, listed at high effort; no Medium mode is listed), or Claude Code with `model:claude-opus-5-5`
- **Workers:** Amp mode `gpt6l` (GPT-6 Luna), or Claude Code with `model:claude-haiku-5-5`
- **Executor:** runner `zains`, one worktree per worker

Selected models shown on screen

Each thread header shows the model used. We read it out loud.

THREE TASKS

Tasks for the workers

TASK	GOAL	KEY FILES
1 · Blog category filters	Filters compose with the existing language routes	`app/models/posts.ts`, `app/components/BlogArchiveList.tsx`, `app/routes/_layout.blog_*.tsx`
2 · Atlas page	Curated topical navigation, inspired by yegge.ai/atlas. Not a graph.	New route; reads post metadata
3 · First-party AMA form	Replace the dead tanyaaja.in link with a question form and private storage	`app/routes/_index.tsx`, new route and API

Cloudflare Worker + D1 is a candidate for task 3. It is not decided and not provisioned.

SHARED CONTRACT

The planner fixes this before any worker starts

- **Category slugs:** one list, owned by the planner.
- **Post shape:** `categories: string[]` in `app/models/posts.ts`.
- **Filter URL:** `?category=<slug>` on `/blog`, `/blog/en`, `/blog/id`.
- **Empty state:** one message per language.

```
// contract v1 · owner: planner
categories: string[]
url: ?category=<slug>
empty: per-language message
```

OWNERSHIP

Who owns which files, ports, and data

THREAD	TASK	OWNS	MUST NOT TOUCH	PORT	TEST DATA
T0 Planner	Contract, integration	contract file, merge	feature code	5173	none
T1 Blog	Category filters	posts model, archive component, blog routes	`_index.tsx`	5174	fixture posts
T2 Atlas	Atlas page	new atlas route	blog routes, posts model	5175	read-only posts
T3 AMA	Question form	new AMA route and API	blog and atlas files	5176	local state dir

Ports are proposals. The planner confirms them before the workers start.

TRADE-OFFS

Worktrees isolate source edits. Nothing else.

WORKTREES ISOLATE	WORKTREES DO NOT ISOLATE
source files	ports and processes
git branches	CPU and RAM
uncommitted edits	browser sessions
	shared databases and external services
	merge conflicts

Orbs isolate each thread's runtime. They still do not fix merge conflicts or shared external services.

Messages carry context, not code

```
planner → T2
Contract v1 is in the contract file. Your port is 5175. Reply to T0 with branch,
changed files, and checks.

T2 → planner
BLOCKED: Atlas needs slug `engineering`. Not in contract v1. Waiting for your decision.

T1 → planner
DONE: branch and file list sent. Checks: report attached.
```

Format examples. They show the message shape, not a result.

One thread can reach several machines

Use **Rex** to reach another machine from the same Amp thread.

Create another thread for **a separate job**, not just machine access.

Keep thread communication for workers that own separate tasks.

```
# routes used in one thread
rex … # local Mac
rex -S vikas-rex …
rex -S zains-rex …
```

Three Macs, one Amp context. Avoid repeated thread setup; commands and output still use tokens.

INTEGRATION

Integrate, then check the combined branch

1. Merge the contract, then T1, T2, T3.
2. Run typecheck and build on the combined branch.
3. Check each category on each language route, with empty states.
4. Check that every Atlas link resolves.
5. Check AMA: valid, empty, too long, rate limit.

Matrix to fill in:

category × language, with pass or fail and the reason.

Small and honest

- **Validation:** required text, length limit, checked on the server.
- **Submission states:** idle, sending, sent, failed.
- **Basic abuse controls:** hidden field, rate limit, size limit.
- **Not included:** public feed, accounts, admin dashboard.

Private is not anonymous. We do not promise absolute anonymity.

Hosting

The site stays on Vercel. A Cloudflare Worker + D1 is only a candidate for this one endpoint.

What Orbs isolate, and what they do not

- Amp-hosted Orbs isolate each thread's runtime environment.
- Orbs do not fix merge conflicts or shared external services.
- Self-hosted Orbs register per workspace. Individuals can create a free workspace.
- Infrastructure costs stay with you.

Personal experience

In Palka I tried self-hosted Orbs. Workspace-scoped setup did not fit my personal workflow. This is my report, not checked here.

Today

No Orbs at all.

EVIDENCE

One usage record per thread

THREAD	MODEL (MODE OR FEATURE)	RUNNER	START → END	USAGE (AMP RECORD)	CHECKS	RESULT
T0 Planner		`zains`				
T1 Blog		`zains`				
T2 Atlas		`zains`				
T3 AMA		`zains`				

Blank until the run. Empty cells are a result too.

References

- Amp — Orbs and Self-Hosted Orbs
- SWE Growth — Coding Agent Benchmark agenda
- Dan Roam — The Pop-Up Pitch (this talk's storyline)
- Yegge Atlas — inspiration for task 2
- Zain Fathoni — You don't need CLI
- Zain's Rex multi-machine thread — skill + CLI example; thread access required

> **Messages carry context.**
>
> **Checks carry proof.**

Q & A

zainfathoni.com/royal-vs-frugal-amp.html

ampcode.com

Zain Fathoni